Task-Aware Harness Provisioning for LLM Agents in Mission-Critical Infrastructure Operations

Liangtao Lin
Nanyang Technological University
Singapore
liangtao002@e.ntu.edu.sg

Qingang Zhang
Nanyang Technological University
Singapore
qingang.zhang@ntu.edu.sg

Zhaomeng Zhu
Nanyang Technological University
Singapore
zhaomeng.zhu@ntu.edu.sg

Tianwei Zhang
Nanyang Technological University
Singapore
tianwei.zhang@ntu.edu.sg

Yonggang Wen
Nanyang Technological University
Singapore
ygwen@ntu.edu.sg

Abstract

LLM agents have been widely adopted to operate mission-critical infrastructure (MCI). These agents normally rely on a *harness* that determines what information they can access, which tools they can use, and what actions they can take. Existing systems often expose the same comprehensive harness to every task, which may not be necessary and cause resource wastes. In this paper, we focus on the identification of optimal harness configurations, and view it as a resource-matching problem between what each task requires and what the harness provides. To measure this match, we classify MCI tasks based on the mathematical representation of the underlying system and rank harness configurations by the amount and type of information they provide. We then construct task-to-harness mappings from two sources: mining research literature and measuring controlled agent execution. Leveraging the measured mapping, we propose a new harness provisioning algorithm: map-guided escalation. It begins with a task-specific harness and expands to full provision only after a failed self-check. We evaluate our method in two representative MCI tasks: in liquid cooling, it improves the agent accuracy from 0.652 under full provision to 0.715 and achieves accuracy comparable to Reflexion with 48% fewer tokens; In power grids, full provision remains accuracy-optimal, while map-based provisioning offers lower-cost alternatives. These findings show that harness provisioning follows a domain-dependent accuracy–cost Pareto frontier rather than a universal optimum.

1 Introduction

Mission-critical infrastructure (MCI), including information technology, power, and water systems, comprises systems and assets whose disruption or destruction could severely affect security, the economy, public health, or public safety [3]. Operating and maintaining (O&M) these systems require continuous monitoring, fault diagnosis, future-state prediction, maintenance planning, and control. Recent studies have explored LLM agents for cloud operations and industrial asset management [19, 28]. To perform these tasks, an agent relies on a *harness* that provides task-relevant information, tools, and system access [15, 22]. It is thus critical to configure the optimal harness for each given task.

We frame this as a resource-matching problem: a harness determines the information and capabilities available to the agent, while each task imposes its own requirements for reliable execution. The simplest one-size-fits-all harness strategy, which provides every task with the same configurations, has been questioned [39]. More advanced solutions are proposed, e.g., adapting the harness by retrieving resources relevant to the current task [24, 29], granting access according to predefined rules [11, 32], or allowing the agent to decide which resources it needs [2, 9]. However, these approaches usually infer what may be useful rather than measure what is actually sufficient. In MCI O&M, provisioning must balance not only task success but also economic, including token usage, latency, and computation, as well as information security by limiting unnecessary data exposure. Too little provision may prevent reliable execution, whereas excessive provision increases cost and exposure. This motivates our central question: *can we determine the minimum harness a given task needs for reliable execution*?

We hypothesize that each task category has a characteristic harness demand, and seek to identify the mapping between them, which we call the *task-to-harness mapping*. To this end, we first need two comparable representations: one for task demand and one for harness provision. MCI O&M tasks are commonly described using labels such as detection, diagnosis, and prediction, but these labels lack consistent definitions across domains and studies, and do not systematically cover the full task space. We therefore derive a new task taxonomy from the mathematical representation of the underlying physical system, yielding categories with explicit and distinguishable boundaries. On the provision side, we separately define a cumulative harness hierarchy according to the amount and type of system information made available to the agent. Together, these representations provide two structured spaces for task types and resource provision.

We propose two complementary strategies to identify the task-to-harness mapping. First, we examine if it is already implicit in prior MCI O&M research. We collect and analyze more than 1,200 papers published over the past decade and find a clear pattern: tasks concerning future outcomes, latent states, or interventions tend to use more extensive harness resources. Second, to verify whether this literature-derived pattern reflects actual agent requirements, we conduct controlled execution experiments in two MCI cases: liquid cooling and power grids. We build simulation-backed agent environments with multiple harness levels and a benchmark of 240 verified tasks. Execution results partly align with the literature-derived pattern but also reveal domain-specific variation. In liquid cooling, several task categories achieve best performance below

full provision while using fewer tokens and shorter time, showing that the most comprehensive harness is not always optimal.

With the task-to-harness mapping, an MCI agent can provision resources according to the type and estimated demand of each atomic task. The simplest solution is to directly retrieve the corresponding harness level from either the literature-derived or execution-derived map, requiring no additional routing call. To account for variation among tasks within the same category, we further propose a *map-guided escalation* strategy: the agent starts with the execution-derived provision and retries with the full harness only when the initial execution fails its self-check. On held-out tasks, execution-map lookup performs comparably to experience-augmented LLM routing while using 14% and 12% fewer tokens than full provision. Map-guided escalation improves accuracy over full provision from 0.652 to 0.715 in liquid cooling. It also achieves comparable accuracy to Reflexion, an iterative method that uses verbal self-reflection on task feedback to improve subsequent attempts [33], while using 48% fewer tokens. In the power-grid domain, full provision remains accuracy-optimal, while execution-map lookup provides a lower-cost operating point. Overall, these results reveal a domain-dependent Pareto frontier between execution accuracy and resource cost rather than a single provisioning policy that is optimal across systems.

Our contributions are summarized as follows:

- We formulate harness provisioning for MCI agents as a resource-matching problem, balancing task performance, execution cost, and unnecessary information exposure.
- We introduce a novel MCI agent task taxonomy derived from system equations and an ordered harness hierarchy with increasing system access, providing comparable representations of task demand and harness provision.
- We construct two task-to-harness maps through data-driven analysis of more than 1,200 MCI papers and controlled execution on a new benchmark of 240 verified tasks in two physical cases.
- We propose map-guided escalation, which uses the measured mapping to initialize harness provision and expands access only when needed, improving the accuracy–cost trade-off over fixed and dynamically routed provisioning.

2 Related Work

2.1 LLM Agents and Benchmarks for MCI

Operational intelligence in mission-critical infrastructure has traditionally relied on task-specific pipelines. Representative KDD systems include EGADS for scalable anomaly detection [13], SR-CNN and OmniAnomaly for time-series monitoring [31, 34], and CIRCA for causal root-cause analysis [16]. These methods provide strong solutions for individual detection or diagnosis tasks, but do not support a general agent across the O&M lifecycle.

Recent work moves toward interactive agents and executable benchmarks. AIOpsLab deploys fault-injected cloud environments and evaluates agents across the incident lifecycle [19], while AssetOpsBench provides tools and scenarios for industrial asset operation and maintenance [28]. In related infrastructure control, LLM-Light evaluates LLM agents for traffic-signal decision making [12]. Existing surveys emphasize that realistic agent evaluation must cover behavior, reliability, safety, environments, and tooling [23]. These benchmarks evaluate agents within predefined environments and interfaces. Our benchmark instead varies harness provision along an ordered ladder, holds the executor fixed, and estimates the sufficient provision for each task class.

2.2 Improving Agent Performance

Most prior work improves the execution side of an agent by changing how it reasons, learns, or reacts within a given environment. ReAct interleaves reasoning with actions [37], Reflexion introduces feedback-driven retries [33], and ExpeL retrieves experience from prior trajectories [41]. Training-based methods further strengthen planning and tool use: AgentGen generates environments and tasks for planning-oriented instruction tuning [8], while Tool-MVR learns tool invocation and error correction from verified trajectories [20]. These methods primarily improve the executor or its execution process under an available harness.

A complementary line of work improves the supply side by adapting what is exposed to the executor. ToolLLM and AnyTool retrieve task-relevant APIs from large tool collections [4, 29], while Instruction-Tool Retrieval (ITR) dynamically retrieves relevant instruction fragments and exposes a reduced tool subset at each step [6]. Chameleon plans compositions of external modules [18], and Sufficient Context predicts whether retrieved textual evidence is adequate for answering a query [10]. Model-routing methods such as RouteLLM and AutoMix similarly allocate computational capacity according to predicted need, although they switch executors rather than vary the harness of a fixed executor [1, 25]. These approaches select resources through relevance, learned routing, or model-reported need. Our work instead measures the lowest harness provision that preserves execution performance.

2.3 Harness Design, Evaluation, and Governance

Recent work increasingly treats the agent harness as a first-class system layer spanning execution control, tool access, context, state, verification, and recovery [15, 22]. Natural-Language Agent Harnesses externalizes control logic into portable specifications [27]. Meta-Harness searches over harness implementations [14]. Harness-Bench measures configuration-level harness effects across models and workflows [38]. These studies establish that agent performance depends on the model–harness configuration, but focus on harness representation, optimization, or cross-configuration evaluation.

A parallel body of work governs how harness resources may be used. Progent enforces programmable tool policies [32], Prompt Flow Integrity constrains information flows [11], and AgentSandbox and MiniScope provide sandboxing and permission analysis [40, 42]. ToolPrivBench studies unnecessary privilege selection [36], while HarnessAudit evaluates boundary compliance over complete execution trajectories [17].Collectively, these studies treat the harness as an object to design, optimize, evaluate, or govern. Our work addresses a complementary question: for a fixed executor and recurring task class, what is the sufficient harness provision that preserves execution performance?

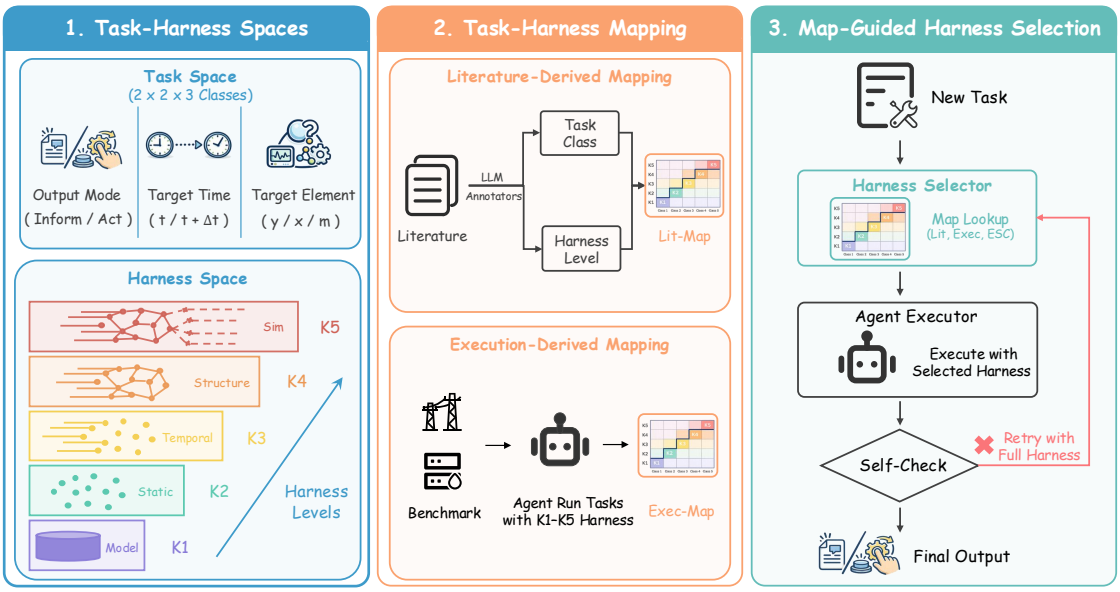

Figure 1: Overview of the Proposed Framework, which defines the task and harness spaces, estimates literature- and execution-derived task-to-harness maps, and uses the resulting map for harness selection with optional self-check-triggered escalation.

3 Overview

Figure 1 presents the overall workflow of our methodology, which consists of three stages: defining the task and harness spaces, estimating the relationship between them, and using the resulting mapping to select harnesses for new tasks.

First, to compare what different tasks require with what different harnesses provide, we formally define the task space from the underlying physical system, and organize harnesses into levels with increasing information and capabilities (Section 4).

Second, we establish the task-to-harness mapping from two perspectives (Section 5). We first mine existing MCI O&M studies to examine which harness levels prior work uses for different task classes. We then construct a controlled benchmark in two MCI environments and execute the same agent under different harness levels to empirically estimate the mapping.

Third, we investigate how the resulting map can be used to select harnesses for new tasks (Section 6). Given a MCI agent task, we first assign it to a task class and retrieve the corresponding harness level from the map. Direct lookup uses this level as the final provision, whereas map-guided escalation uses it as the initial provision and retries with the full harness if the first execution fails its self-check. This allows the class-level map to guide efficient provisioning while accounting for variation among individual tasks.

4 Task–Harness Spaces

Determining which harness a task requires is fundamentally a measurement problem. Before this relationship can be measured, task demand and harness provision must be expressed using stable and comparable representations. We therefore construct two rulers. The first organizes MCI tasks according to what they query or affect in the underlying physical system. The second organizes harness configurations according to the information, tools, and operational access provided to the agent. Together, these rulers turn harness provisioning into a measurable mapping from task class to minimum sufficient harness.

4.1 Task Space

Conventional task labels such as detection, diagnosis, prediction, and planning are unsuitable as primary task categories because their meanings vary across domains, applications, and implementations. We therefore derive the task space directly from a common representation of partially observed physical systems.

Let y_t denote observable signals, x_t denote latent physical states, and m_t denote the system structure and governing mechanisms, including topology, dynamics, constraints, and control logic. We abstract the system as

$$x_{t+1} = F_{m_t}(x_t, a_t, d_t) + \xi_t, \qquad y_t = G_{m_t}(x_t) + \epsilon_t,$$

where a_t is an intervention, d_t is an external disturbance, and ξ_t and ϵ_t denote process and observation uncertainty. This representation separates what is directly observed, what remains latent, and what governs system behavior. These three elements provide the possible targets of an MCI task.

Then we represent each task as

$$T = \langle \omega, \tau, e \rangle, \quad \omega \in \{\text{Inform}, \text{Act}\}, \tau \in \{t, t+\Delta\}, e \in \{y, x, m\}.$$

The target element e specifies whether the task concerns observable signals, latent states, or system mechanisms. The target time τ distinguishes the current system from a future outcome. The output mode ω specifies whether the agent should report information about the target or produce an intervention intended to affect it. Their Cartesian product yields $2 \times 2 \times 3 = 12$ canonical task classes.

Given the evidence B_t available at task time, the required output takes one of two forms:

$$o_T = \begin{cases} \hat{r}_T = \rho_T(p(e_\tau \mid B_t)), & \omega = \text{Inform}, \\ a^\star = \arg\min_{a \in \mathcal{A}} \mathbb{E}\left[J_T\left(e_\tau^{do(a)}\right) \mid B_t\right], & \omega = \text{Act}. \end{cases}$$

Here, $p(e_\tau \mid B_t)$ represents the agent's inferred belief about the target, and ρ_T converts this belief into the task-specific report $\hat{r}_T$, such as a value, label, event, or explanation. For an Act task, $e_\tau^{do(a)}$ denotes the target outcome under intervention a, and J_T evaluates the desirability of that outcome. An Inform task therefore reports knowledge about e at time τ, whereas an Act task selects an intervention according to its expected effect on e at that time.

Conventional task types can be represented by specific coordinates in this space according to their target, time horizon, and output mode. For example, detection corresponds to $\langle \text{Inform}, t, y \rangle$, diagnosis to $\langle \text{Inform}, t, m \rangle$, and prediction to $\langle \text{Inform}, t + \Delta, y \rangle$. In the following, we use the three-dimensional representation $\langle \omega, \tau, e \rangle$ to denote task types.

4.2 Harness Space

A task representation specifies what must be accomplished, but not what information, models, tools, and interfaces are exposed to the executor. Motivated by prior work that organizes system information by temporal reach and structural scope [5, 30], we define five cumulative levels of harness provision:

$$\mathcal{H}_{K1} \subset \mathcal{H}_{K2} \subset \mathcal{H}_{K3} \subset \mathcal{H}_{K4} \subset \mathcal{H}_{K5}.$$

- $K1$**: Model-only reasoning.** The agent receives only the task prompt and relies on its parametric knowledge.
- $K2$**: Static knowledge.** The harness additionally provides fixed resources such as manuals, SOPs, specifications, design documents, and rule bases.
- $K3$**: Temporal observations.** The harness additionally provides historical or real-time telemetry, logs, alarms, and time-series measurements.
- $K4$**: Structure and physics.** The harness additionally provides topology, component relations, governing equations, physical constraints, and control logic.
- $K5$**: Forward simulation.** The harness additionally provides executable simulation, digital-twin rollouts, counterfactual evaluation, and optimization over hypothetical future trajectories.

The ordering reflects increasing access to the physical system rather than increasing intrinsic intelligence of the executor. It also does not impose a fixed mapping from task coordinates to harness levels: a future-facing task may be solved from temporal observations, while a current-state task may require structural models or simulation. Although provision increases from $K1$ to $K5$, performance need not improve monotonically because additional context and tools may increase cost, introduce irrelevant evidence, or create unnecessary action opportunities. Thus, $K5$ denotes maximal provision rather than an assumed performance optimum.

5 Estimating the Task-to-Harness Map

The task space characterizes what different MCI tasks demand, while the harness space characterizes what information and capabilities are provided to the agent. Our goal is to identify the relationship between these two spaces by assigning a harness level to each task class. We first define a common mapping rule and then estimate the map from two sources. The literature-derived map summarizes which harness levels prior MCI O&M studies use for different task classes, whereas the execution-derived map identifies the lowest harness level that achieves near-best performance when the same agent is evaluated under different harness levels.

5.1 Mapping Definition

Let $A_s(T, K)$ denote the support for assigning harness $\mathcal{H}_K$ to task class T under evidence source $s \in \{\text{lit}, \text{exec}\}$. We define

$$\mathcal{G}_s(T) = \min\left\{K \in \mathcal{K} : A_s(T, K) \geq \max_{K' \in \mathcal{K}} A_s(T, K') - \epsilon_s\right\},$$

where $\epsilon_s \geq 0$ is a source-specific tolerance. The mapping selects the lowest harness level whose support is within ϵ_s of the maximum for that task class, avoiding unnecessary provision when several levels receive similar support.

For the literature-derived map, we set $A_{\text{lit}}(T, K) = P_{\text{lit}}(K \mid T)$, where $P_{\text{lit}}(K \mid T)$ is the observed frequency of harness level K among prior studies assigned to task class T. The resulting map $\mathcal{G}_{\text{lit}}$ summarizes how prior work has provisioned different task classes.

For the execution-derived map, we set $A_{\text{exec}}(T, K) = \mu_{\text{exec}}(T, K)$, where $\mu_{\text{exec}}(T, K)$ is the mean execution score obtained under harness level K for task class T. The resulting map $\mathcal{G}_{\text{exec}}$ selects the lowest harness level whose measured performance is within ϵ_{exec} of the best observed performance.

5.2 Literature-Derived Mapping

We first examine which harness levels existing MCI O&M studies use for different task classes. We collect approximately 2,000 candidate papers from *arXiv*, *Semantic Scholar*, and *OpenAlex*, and retain more than 1,200 studies within our scope. For each paper p, three LLM annotators independently extract its primary task class $T_p \in \mathcal{T}$ and the highest harness level $K_p \in \mathcal{K}$ materially used by the proposed method. The labels are consolidated by majority vote, with unresolved disagreements manually adjudicated.[1]

We then estimate

$$P_{\text{lit}}(K \mid T) = \frac{N_{\text{lit}}(T, K)}{\sum_{K' \in \mathcal{K}} N_{\text{lit}}(T, K')},$$

where $N_{\text{lit}}(T, K)$ is the paper count associated with task class T and harness level K. Applying the common mapping rule produces $\mathcal{G}_{\text{lit}}$.

The above map captures established provisioning practice rather than verified sufficiency. A frequently-used harness level may still be unnecessary, insufficient, or over-provisioned for a particular agent or environment. Nevertheless, it provides a literature-scale view of how harness provision varies across task classes.

[1] Appendix A details the collection, screening, and annotation procedures.

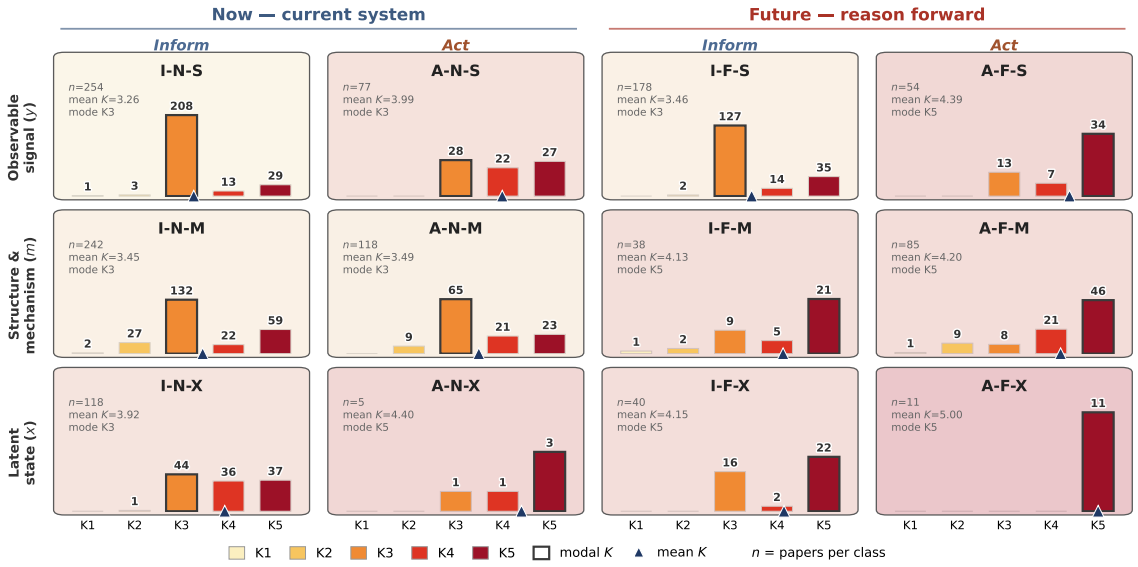

Figure 2: Literature-derived Harness Distributions across the 12 Task Classes. Based on 1,220 MCI O&M papers, each panel shows the number of papers assigned to harness levels K_1–K_5 for one task class; the outlined bar marks the modal level while the triangle and background marks the mean. The distributions generally shift toward higher harness levels for Act, Future, and latent-state tasks, while most classes peak at either K_3 or K_5.

5.3 Execution-Derived Mapping

We next measure the relationship directly by evaluating the same agent under different harness levels[2]. We construct two simulation-backed environments: a thermal–hydraulic digital twin of a liquid-cooled data hall and a modified IEEE-14 power-grid environment built with Grid2Op and pandapower [21, 35]. Each environment generates replayable system trajectories from which we instantiate ten tasks for each of the 12 task classes, yielding 240 tasks in total. The tasks cover reporting current observations, inferring latent states or system mechanisms, predicting future outcomes, and selecting interventions. Each task is paired with a deterministic simulator-backed oracle and undergoes both programmatic and human verification.[3]

Every task is executed under all five harness levels using the same agent, reasoning protocol, and evaluation procedure; only the available harness provision varies. Let $\mu_{\text{exec}}(T, K)$ denote the mean score of harness level K over tasks in class T. Applying the common mapping rule gives

$$\mathcal{G}_{\text{exec}}(T) = \min\left\{K \in \mathcal{K} : \mu_{\text{exec}}(T, K) \geq \max_{K' \in \mathcal{K}} \mu_{\text{exec}}(T, K') - \epsilon_{\text{exec}}\right\}.$$

The above execution-derived map identifies the lowest harness level that achieves near-best performance when the same agent is evaluated across harness levels.

6 Map-Guided Harness Selection

Given an atomic task q, the system first assigns it to a task class $T(q)$ and uses the task-to-harness map to select an initial harness. The executor then completes the task using only the information and capabilities exposed by the selected harness. Because tasks within the same class may still differ in difficulty and required information, we allow the harness to expand to K_5 when the initial execution fails its self-check.

Initial harness selection. Since the task-to-harness map captures recurring provisioning patterns for each task class, we use the mapped level as a class-level prior for selecting the initial harness. Given a task q and a map $\mathcal{G}_s$, where $s \in \{\text{lit}, \text{exec}\}$ denotes the map source, the initial harness level is

$$K_0(q; s) = \mathcal{G}_s(T(q)).$$

Once the task class $T(q)$ is available, this step requires only a deterministic table lookup and introduces no additional routing call. Let π denote the fixed executor. Running π under $\mathcal{H}_{K_0(q;s)}$ produces

$$(o_0, c_0, r_0) = \pi\big(q, \mathcal{H}_{K_0(q;s)}\big),$$

where o_0 is the initial output, $c_0 \in \{\text{pass}, \text{fail}\}$ is its self-check result, and r_0 contains findings that can be reused in a subsequent attempt.

Conditional escalation. The mapped level is a class-level prior and may not provide sufficient information or capabilities for every task instance. When the initial execution fails its self-check, we therefore expand the harness to K_5, exposing the executor to the full set of information, tools, and operational permissions. The initial result is accepted if it passes the self-check or if the selected harness is already K_5; otherwise, the executor retries once under K_5 while retaining the useful findings r_0 from the initial attempt. The final harness and output are

$$\big(K_{\text{final}}(q), o(q)\big) = \begin{cases} \big(K_0(q; s), o_0\big), & c_0 = \text{pass or } K_0(q; s) = K_5, \\ \big(K_5, \pi(q, \mathcal{H}_{K_5}; r_0)\big), & c_0 = \text{fail and } K_0(q; s) < K_5. \end{cases}$$

[2]Appendix C details the domain-specific harness implementation.
[3]Appendix B details the benchmark construction and verification.

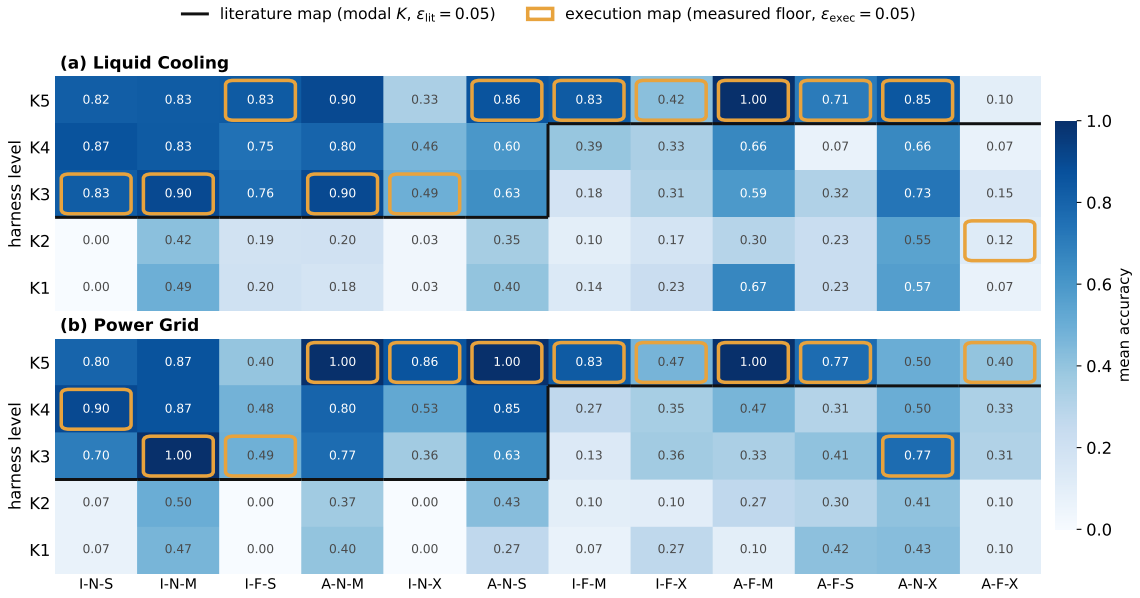

Figure 3: Task-to-Harness maps. Cells report the mean construction-split accuracy for each task class and harness level (5 tasks per class, 3 runs). Amber boxes mark the lowest level within $\epsilon_{exec} = 0.05$ of the best execution score, while the black staircase marks the literature-derived level. Columns are ordered by the literature mean harness level.

Method variants. This formulation produces three evaluated variants. Lit-Lookup uses $\mathcal{G}_{lit}$ and disables escalation, while Exec-Lookup uses $\mathcal{G}_{exec}$ and likewise accepts the mapped harness as final. Map-ESC uses $\mathcal{G}_{exec}$ for initial selection and applies the self-check-triggered escalation rule above. The two lookup variants isolate the value of the maps themselves, while Map-ESC evaluates whether failures caused by instance-level variation can be recovered through conditional expansion.

We escalate directly to K_5 rather than testing each intermediate level to avoid repeated execution overhead; Appendix H.2 shows that intermediate retries rarely terminate before reaching K_5 in the evaluated domains. The executor and reasoning protocol remain fixed across both attempts, and the harness enforces the capability boundary at each level. Before escalation, the executor therefore cannot access information, tools, or permissions beyond its initially selected harness.

7 Experiments

Having defined the task-to-harness maps and their deployment policies, we now evaluate whether the proposed structure is empirically supported and operationally useful. Our experiments address four research questions.

- **RQ1**: Do task classes exhibit distinct harness requirements? Does richer provision consistently improve performance?
- **RQ2**: How reliably can the task-to-harness map be estimated from literature and execution? How do the two evidence sources compare with each other?
- **RQ3**: How should the map be operationalized at deployment time: through direct lookup or map-guided escalation?
- **RQ4**: Whether do the resulting task-to-harness relationships generalize across domains and executor models? What is the impact of the granularity, robustness, and deployment implications of the learned map?

7.1 Experimental Setup

Environments and tasks. We evaluate two simulation-backed MCI domains: a proprietary liquid-cooling digital twin (Liquid) and a power-grid environment built with Grid2Op and pandapower (Grid). Each domain contains 120 tasks, with ten instances for each of the 12 classes $T = \langle \omega, \tau, e \rangle$. Within each class, five tasks are used for map construction and five form a disjoint test split. All 240 tasks undergo programmatic and human verification. We perform the full K_1–K_5 sweep on all tasks, but construct the execution-derived map only from the construction split and evaluate provisioning policies on the test split.

Execution protocol. We instantiate the five cumulative harness levels defined in Section 4. Unless otherwise stated, all conditions use the same frozen GPT-5.4 [26] executor with a ReAct loop, at most 20 tool iterations, and a 300-second timeout; only harness provisioning varies. Both maps use a tolerance of $\epsilon_{lit} = \epsilon_{exec} = 0.05$.[4]

Compared policies. We compare maximal provisioning (Full-K_5), relevance-based retrieval (ITR [6]), LLM routing with and without prior experience (LLM-Route, LLM+Exp), a learned cascade (AutoMix [1]), and progressive escalation from K_1 (Blind-ESC). Our policies include direct lookup from the literature and execution

[4]Appendix E reports tolerance sensitivity, selected-level stability, statistical tests, and oracle definitions.

Table 1: Harness Provisioning on the Test Split. All policies use the same frozen executor and differ only in how the harness is selected. Latency and tokens are normalized to Full-K5. Bold and underline denote the best and second-best deployable results in each column.

			Liquid			Grid		
Group	Policy	Basis	Acc. ± std	Lat.	Tok.	Acc. ± std	Lat.	Tok.
Fixed	Full-K5	maximal provision	0.652 ± 0.001	1.00×	1.00×	**0.806 ± 0.009**	**1.00**×	<u>1.00×</u>
Routing	ITR	semantic relevance	<u>0.670 ± 0.015</u>	0.90×	0.99×	0.780 ± 0.010	1.20×	1.02×
	LLM-Route	LLM judgment	0.626 ± 0.021	**0.74**×	0.93×	0.744 ± 0.024	1.29×	1.03×
	LLM+Exp	judgment with experience	0.655 ± 0.010	<u>0.75×</u>	0.96×	0.766 ± 0.011	1.26×	1.09×
Cascades	AutoMix	trained check from K3	0.639 ± 0.011	1.08×	1.94×	<u>0.785 ± 0.029</u>	3.03×	2.61×
	Blind-ESC	self-check from K1	0.664 ± 0.026	0.90×	1.33×	0.742 ± 0.020	5.33×	1.75×
Ours	Lit-Lookup	literature prior	0.658 ± 0.017	0.84×	<u>0.89×</u>	0.664 ± 0.018	1.30×	1.01×
	Exec-Lookup	execution-derived map	<u>0.670 ± 0.020</u>	0.83×	**0.86**×	0.762 ± 0.023	<u>1.13×</u>	**0.88**×
	Map-ESC	execution map with self-check	**0.715 ± 0.014**	1.03×	1.15×	0.782 ± 0.010	1.37×	1.03×
Reference	Class Oracle	expected static ceiling	*0.694*	—	—	*0.809*	—	—
	Task Oracle	expected static ceiling	*0.783*	—	—	*0.843*	—	—

maps (Lit-Lookup, Exec-Lookup) and two-stage map-guided escalation from the execution-derived mapped level (Map-ESC). We separately compare Map-ESC with Reflexion [33] and ExpeL [41], which adapt execution under a fixed K_5 harness.[5]

Metrics. Task performance is scored in $[0, 1]$ using a Gemini-3.1-Pro [7] judge combined with rule-based checks. Each condition is run three times, and results are reported as mean ± population standard deviation. Token counts include routing calls and all escalation attempts; latency and tokens are normalized to Full-K_5. We report two non-deployable references: the *Class Oracle* selects the highest-scoring harness for each task class, while the *Task Oracle* selects the highest-scoring harness separately for each task, both using mean scores across the three runs. [6]

7.2 RQ1: Do Task Classes Require Different Harnesses?

Figure 2 shows that harness provision in prior MCI O&M studies varies systematically across task classes. Among 1,200+ papers, current Inform tasks targeting observable signals or system mechanisms are concentrated at K_3, whereas Future, Act, and latent-state tasks shift more strongly toward K_5. This indicates two recurring provisioning patterns in prior work: observation-driven provision centred on K_3 and simulation-backed provision centred on K_5.

Figure 3 provides direct execution evidence by evaluating the same executor under K_1–K_5. [7] The lowest level achieving performance within $\epsilon = 0.05$ of the best class score varies from K_2 to K_5 in Liquid and from K_3 to K_5 in Grid. Performance is also non-monotonic: five classes in Liquid and four in Grid attain their highest score below K_5. Thus, harness requirements vary across both task classes and domains, and maximal provision is neither uniformly necessary nor uniformly optimal.

7.3 RQ2: How Should the Task-to-Harness Map Be Estimated?

Having established that harness requirements vary across task classes, we next compare the two sources used to estimate the map. As shown in Table 1, Lit-Lookup selects the level most frequently observed in prior studies, whereas Exec-Lookup selects the lowest level whose construction-split performance is within $\epsilon_{\text{exec}} = 0.05$ of the best level for each task class. The two maps perform similarly on Liquid (0.658 versus 0.670), but differ substantially on Grid, where Exec-Lookup achieves 0.762 compared with 0.664. Literature evidence therefore provides a useful prior, while execution evidence better adapts the map to the evaluated domain and executor.

Exec-Lookup also provides a stronger accuracy–efficiency balance than per-task routing. Directly asking an LLM to select the harness (LLM-Route) yields lower accuracy in both domains (0.626 and 0.744). Providing the router with a playbook distilled from construction-split trajectories (LLM+Exp) improves its decisions (0.655 and 0.766), but does not consistently outperform Exec-Lookup and incurs an additional routing call. Retrieving resources according to their semantic relevance to the task (ITR) achieves comparable accuracy (0.670 and 0.780) but remains close to Full-K_5 in token usage (0.99× and 1.02×), compared with 0.86× and 0.88× for Exec-Lookup. Semantic relevance can therefore identify potentially useful resources, but does not directly determine a cost-efficient harness. Overall, measured class-level execution evidence provides the most consistent basis for initial harness selection.

7.4 RQ3: How Should the Map Guide Harness Selection?

The task-to-harness map supports two deployment modes: using the mapped harness directly or using it as the initial provision before conditional escalation. As shown in Table 1 and Figure 4, Exec-Lookup achieves accuracies of 0.670 on Liquid and 0.762

[5] Appendix F details the implementation and adaptation of all compared policies.
[6] Appendix D details the executor settings, scoring rules, judge configuration, cost accounting, and repetition protocol.
[7] Appendix G details the aggregate and per-class K_1–K_5 sweeps, split-specific analyses, and absolute execution costs.

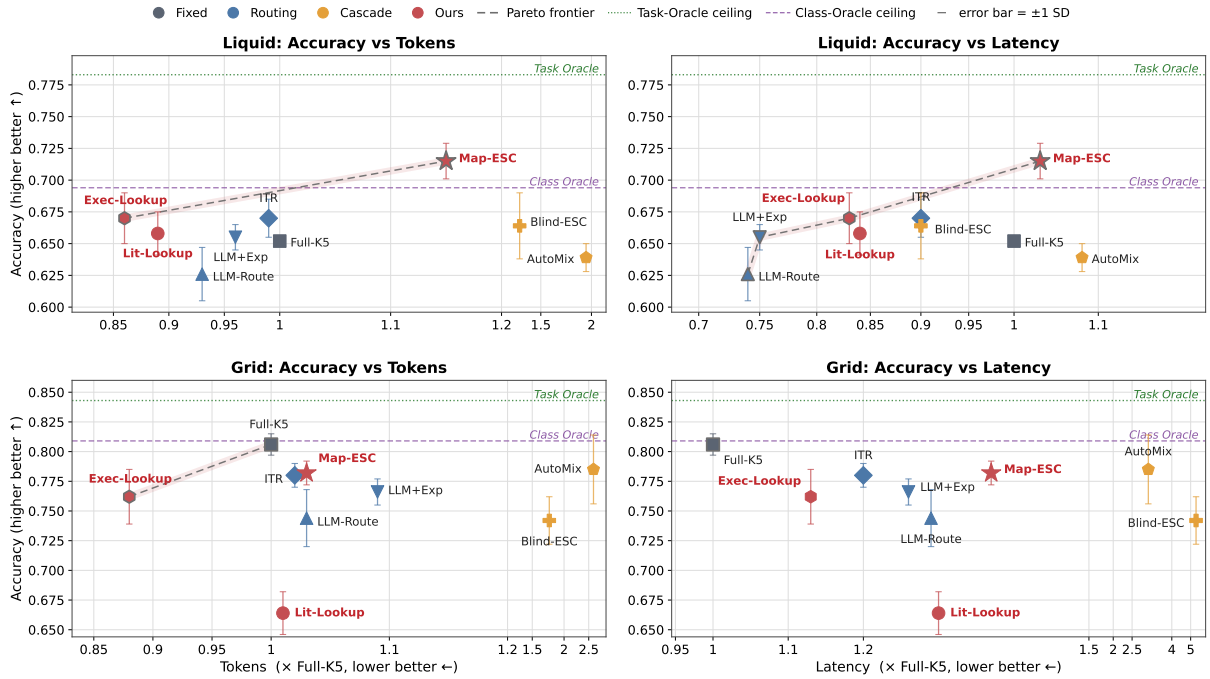

Figure 4: Accuracy–Cost Trade-offs of Harness Provisioning Policies on the Test Split. Latency and tokens are normalized to Full-K_5; dashed lines show the empirical Pareto frontiers.

on Grid, while reducing token usage to 0.86× and 0.88× relative to Full-K_5. Map-ESC retries under K_5 when the initial execution fails its self-check, increasing accuracy to 0.715 and 0.782, with token costs of 1.15× and 1.03×, respectively. Direct lookup therefore favors efficiency, while conditional escalation improves accuracy at additional cost.

Blind-ESC and AutoMix represent two alternative escalation strategies. Blind-ESC starts from K_1 and progressively expands the harness based on self-checks, whereas AutoMix uses a learned cascade to determine whether further provision is needed. Map-ESC instead selects the initial harness from the execution-derived map and allows at most one fallback to K_5. On Liquid, Map-ESC outperforms both Blind-ESC (0.715 versus 0.664) and AutoMix (0.715 versus 0.639). On Grid, it outperforms Blind-ESC (0.782 versus 0.742) and nearly matches AutoMix (0.782 versus 0.785), while using fewer tokens than both (1.03× versus 1.75× and 2.61×). Map-guided two-stage escalation therefore provides the most consistent accuracy–efficiency trade-off across the two domains.

Figure 4 further shows that choosing an appropriate harness configuration depends on both the domain and the deployment objective. On Liquid, Exec-Lookup provides a low-cost configuration, while Map-ESC achieves the highest accuracy among the harness-selection methods. On Grid, Full-K_5 remains the most accurate configuration, whereas Exec-Lookup offers a lower-cost Pareto-efficient alternative. The map therefore helps determine when additional provision improves performance and when it only increases cost.

Table 2 compares Map-ESC with execution-side methods operating under a fixed K_5 harness. Map-ESC matches Reflexion in accuracy (0.715 versus 0.711) while using 48% fewer tokens. ExpeL improves accuracy by 0.016 but requires 44% more tokens. Harness provisioning and execution-side adaptation are therefore complementary, with measured provisioning offering the stronger efficiency trade-off in this setting.[8]

Table 2: Provisioning Versus Execution-side Adaptation on the Liquid Test Split. Tokens are normalized to Map-ESC.

Method	Adaptation lever	Acc. ± std	Tok.
Map-ESC	harness provisioning	0.715 ± 0.014	**1.00**×
Reflexion	retry and reflection	0.711 ± 0.027	1.91×
ExpeL	trajectory retrieval	**0.731 ± 0.009**	1.44×

7.5 RQ4: Does the Map Generalize?

The two domains exhibit the same qualitative need for task-aware provisioning but different deployment outcomes. As shown in Table 1 and Figure 4, task-aware harness selection improves both accuracy and cost in Liquid, whereas Full-K_5 remains accuracy-optimal in Grid and Exec-Lookup provides a lower-cost Pareto-efficient alternative. Task-dependent harness demand therefore persists across domains, although the benefit of reducing provision is domain-specific.

[8] Appendix H details routing-granularity results, escalation diagnostics, experience ablations, and qualitative error analysis.

We further transfer the execution-derived map estimated with GPT-5.4 to Qwen3.5-27B without recalibration. As shown in Table 3, the transferred map improves Qwen over its Full-K_5 baseline on LIQUID (0.726 versus 0.706) and remains close on GRID (0.797 versus 0.808). In contrast, a map estimated from a single Qwen run performs substantially worse in both domains. The task-to-harness relationship therefore transfers across executors, while reliable map estimation benefits from stronger and repeated measurements.

Table 3: Cross-executor Transfer of the Execution-derived Map. All conditions use Qwen3.5-27B as the executor.

Policy	Map source	LIQUID	GRID
Full-K_5	none	0.706	**0.808**
Exec-Lookup	GPT-5.4	**0.726**	0.797
Exec-Lookup	Qwen	0.626	0.762

8 Conclusion

We formulate harness provisioning as an explicit and measurable deployment decision for LLM agents in mission-critical infrastructure. A task taxonomy derived from the mathematical formulation of physical systems characterizes task demand, while a cumulative K_1–K_5 hierarchy characterizes the information and capabilities available to the agent. We estimate the relationship between them using a literature-derived map of prior provisioning practices and an execution-derived map that selects the lowest harness level achieving near-best class-level performance. These maps guide either direct lookup or self-check-triggered escalation to K_5. Our results show that suitable harness configurations vary across task classes and domains, and that additional provision may improve, leave unchanged, or reduce performance. They support selecting harness configurations according to task and domain requirements rather than granting maximal information, tools, and operational access by default.

Limitations and Future Work. The measured task-to-harness relationship is calibrated to the evaluated executor, harness implementation, protocol, and tolerance. Its generalization to new domains, models, and harness implementations requires further validation and, where necessary, recalibration. Our evaluation covers 240 tasks in two simulation-backed environments, which do not fully capture live conditions such as sensor uncertainty, distribution shift, organizational procedures, and human approval. The literature-derived map may reflect publication and annotation bias, while the execution-derived map remains subject to finite-sample variation, judge error, and imperfect self-checks. Future work will expand and release a broader benchmark for LLM agents in MCI O&M, together with reproducible test environments covering additional domains, tasks, disturbances, and tool interfaces. We will also investigate learning-based methods that use the task-to-harness map to guide harness selection and escalation, and extend the evaluation framework to include security and operational risks.

References

[1] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. 2024. AutoMix: automatically mixing language models. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 4164, 35 pages.

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, Vol. 2024. 9112–9141.

[3] Cybersecurity and Infrastructure Security Agency. [n. d.]. Critical Infrastructure Security and Resilience. https://www.cisa.gov/topics/critical-infrastructure-security-and-resilience.

[4] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 470, 18 pages.

[5] Mica R Endsley. 1995. Toward a Theory of Situation Awareness in Dynamic Systems. *Human Factors: The Journal of the Human Factors and Ergonomics Society* 37, 1 (1995), 32–64.

[6] Uria Franko. 2025. Dynamic system instructions and tool exposure for efficient agentic LLMs. *arXiv preprint arXiv:2602.17046* (2025).

[7] Google DeepMind. 2026. Gemini 3.1 Pro: Model Card. https://deepmind.google/models/model-cards/gemini-3-1-pro/.

[8] Mengkang Hu, Pu Zhao, Can Xu, Qingfeng Sun, Jian-Guang Lou, Qingwei Lin, Ping Luo, and Saravan Rajmohan. 2025. AgentGen: Enhancing Planning Abilities for Large Language Model based Agent via Environment and Task Generation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 496–507. doi:10.1145/3690624.3709321

[9] Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 7969–7992. doi:10.18653/v1/2023.emnlp-main.495

[10] Hailey Joren, Jianyi Zhang, Chun-Sung Ferng, Da-Cheng Juan, Ankur Taly, and Cyrus Rashtchian. 2025. Sufficient context: A new lens on retrieval augmented generation systems. In *International Conference on Learning Representations*, Vol. 2025. 20310–20334.

[11] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt flow integrity to prevent privilege escalation in llm agents. *arXiv preprint arXiv:2503.15547* (2025).

[12] Siqi Lai, Zhao Xu, Weijia Zhang, Hao Liu, and Hui Xiong. 2025. LLMLight: Large Language Models as Traffic Signal Control Agents. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 2335–2346. doi:10.1145/3690624.3709379

[13] Nikolay Laptev, Saeed Amizadeh, and Ian Flint. 2015. Generic and Scalable Framework for Automated Time-series Anomaly Detection. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1939–1947.

[14] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052* (2026).

[15] Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Jihun Hamm, Tianyang Wang, and Chandan Reddy. 2026. Agent Harness Engineering: A Survey. https://openreview.net/pdf?id=eONq7FdiHa

[16] Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2022. Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) *(KDD '22)*. Association for Computing Machinery, New York, NY, USA, 3230–3240. doi:10.1145/3534678.3539041

[17] Chengzhi Liu, Yichen Guo, Yepeng Liu, Yuzhe Yang, Qianqi Yan, Xuandong Zhao, Wenyue Hua, Sheng Liu, Sharon Li, Yuheng Bu, et al. 2026. Auditing agent harness safety. *arXiv preprint arXiv:2605.14271* (2026).

[18] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: plug-and-play compositional reasoning with large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1882, 32 pages.

[19] Minghua Ma, Jackson Clark, and Shenglin Zhang. 2025. AIOpsLab in Action: An Open Platform for AIOps Research. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim,

Trondheim, Norway) *(FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 1223–1227. doi:10.1145/3696630.3728619

[20] Zhiyuan Ma, Jiayu Liu, Xianzhen Luo, Zhenya Huang, Qingfu Zhu, and Wanxiang Che. 2025. Advancing Tool-Augmented Large Language Models via Meta-Verification and Reflection Learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 2078–2089. doi:10.1145/3711896.3736835

[21] Antoine Marot, Benjamin Donnot, Gabriel Dulac-Arnold, Adrian Kelly, Aidan O'Sullivan, Jan Viebahn, Mariette Awad, Isabelle Guyon, Patrick Panciatici, and Camilo Romero. 2021. Learning to run a power network challenge: a retrospective analysis. In *NeurIPS 2020 competition and demonstration track*. PMLR, 112–132.

[22] Qianyu Meng, Yanan Wang, Liyi Chen, Yihang Li, Wei Wu, Wenyuan Jiang, Qimeng Wang, Chengqiang Lu, Yan Gao, Yi Wu, et al. 2026. Agent harness for large language model agents: A survey. (2026).

[23] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 6129–6139. doi:10.1145/3711896.3736570

[24] Sarat Mudunuri, Jian Wan, Ally Qin, and Srinivasan Manoharan. 2026. Semantic Tool Discovery for Large Language Models: A Vector-Based Approach to MCP Tool Selection. *arXiv preprint arXiv:2603.20313* (2026).

[25] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665 [cs.LG] https://arxiv.org/abs/2406.18665

[26] OpenAI. 2026. Introducing GPT-5.4. https://openai.com/index/introducing-gpt-5-4/.

[27] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723* (2026).

[28] Dhaval Patel, Chathurangi Shyalika, Suryanarayana Reddy Yarrabothula, Ling Yue, Shuxin Lin, Nianjun Zhou, and James Rayfield. 2026. Results and Retrospective Analysis of the CODS 2025 AssetOpsBench Challenge. *arXiv preprint arXiv:2605.08518* (2026).

[29] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, Vol. 2024. 9695–9717.

[30] Jens Rasmussen. 1985. The role of hierarchical knowledge representation in decisionmaking and system management. *IEEE Transactions on systems, man, and cybernetics* 2 (1985), 234–243.

[31] Hansheng Ren, Bixiong Xu, Yujing Wang, Chao Yi, Congrui Huang, Xiaoyu Kou, Tony Xing, Mao Yang, Jie Tong, and Qi Zhang. 2019. Time-Series Anomaly Detection Service at Microsoft. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. Association for Computing Machinery, New York, NY, USA, 3009–3017. doi:10.1145/3292500.3330680

[32] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable privilege control for llm agents. *arXiv e-prints* (2025), arXiv–2504.

[33] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 377, 19 pages.

[34] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. 2019. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. Association for Computing Machinery, New York, NY, USA, 2828–2837. doi:10.1145/3292500.3330672

[35] L. Thurner, A. Scheidler, F. Schafer, J. H. Menke, J. Dollichon, F. Meier, S. Meinecke, and M. Braun. 2018. pandapower - an Open Source Python Tool for Convenient Modeling, Analysis and Optimization of Electric Power Systems. *IEEE Transactions on Power Systems* (2018). doi:10.1109/TPWRS.2018.2829021

[36] Kaiyue Yang, Yuyan Bu, Jingwei Yi, Yuchi Wang, Biyu Zhou, Juntao Dai, Songlin Hu, and Yaodong Yang. 2026. When Lower Privileges Suffice: Investigating Over-Privileged Tool Selection in LLM Agents. *arXiv preprint arXiv:2606.20023* (2026).

[37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[38] Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, et al. 2026. HarnessBench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922* (2026).

[39] Jianxiang Yu, Jiapeng Zhu, Bochen Lin, Qier Cui, Zichen Ding, and Xiang Li. 2026. Skill is Not One-Size-Fits-All: Model-Aware Skill Alignment for LLM Agents. *arXiv preprint arXiv:2605.30723* (2026).

[40] Kaiyuan Zhang, Zian Su, Pin-Yu Chen, Elisa Bertino, Xiangyu Zhang, and Ninghui Li. 2025. LLM Agents Should Employ Security Principles. *arXiv preprint arXiv:2505.24019* (2025).

[41] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'24/IAAI'24/EAAI'24)*. AAAI Press, Article 2188, 11 pages. doi:10.1609/aaai.v38i17.29936

[42] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G Patil, Vivian Fang, and Raluca Ada Popa. 2025. MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents. *arXiv preprint arXiv:2512.11147* (2025).

A Literature Corpus and Annotation

A.1 Corpus Collection and Screening

We survey how the last decade (2016–2026) of MCI operation-and-maintenance research provisions system access, covering 13 of the 16 CISA critical-infrastructure sectors[9] (Defense Industrial Base, Financial Services, and Government Facilities are out of scope). Candidates come from three public indices, arXiv, Semantic Scholar, and OpenAlex, queried with sector vocabulary only ("substation", "chiller plant", . . .): no task or method words, so the task and harness distributions emerge from the papers rather than from the query set. The collection and screening pipeline was built and operated with Claude Code (Fable 5). From roughly 2,000 crawled candidates, an LLM screening agent triages title and abstract, discarding surveys, position papers, and papers without a primary MCI O&M task, and full-text extraction runs on the survivors. The screened corpus contains 1,223 papers, of which 1,220 receive complete task-coordinate and harness-level annotations and are used to estimate the literature-derived map; Figure 5 shows its sector distribution and vocabulary.

A.2 Task and Harness Annotation Schema

Each paper receives two annotations from separate model calls that never see each other's output, a task coordinate and a harness level, so the class label cannot leak into the harness label. The task call is method- and name-blind: it returns the paper's primary task as one method-free sentence plus only the three axis labels of the taxonomy; no class names appear anywhere in the prompt. The harness call assigns the highest level the proposed solution *materially* uses, considering both what the deployed method consumes at run time and what building it required, a policy trained in a simulator is K5 even if inference reads only telemetry. Figure 6 summarizes both rubrics in the paper's vocabulary.

A.3 Multi-LLM Annotation Protocol

We use three annotators from different model providers, including `gemini-3.1-pro`, `gpt-5.4-mini`, and `glm-5.2`, to independently assign each paper a task coordinate $\langle \omega, \tau, e \rangle$ and a harness level K, together with supporting evidence from the paper. The task axes are consolidated separately by majority vote, while the harness level is consolidated as a single ordinal label. Majority voting directly resolves 89% of task-axis labels and 97% of harness labels.

[9] https://www.cisa.gov/topics/critical-infrastructure-security-and-resilience/critical-infrastructure-sectors

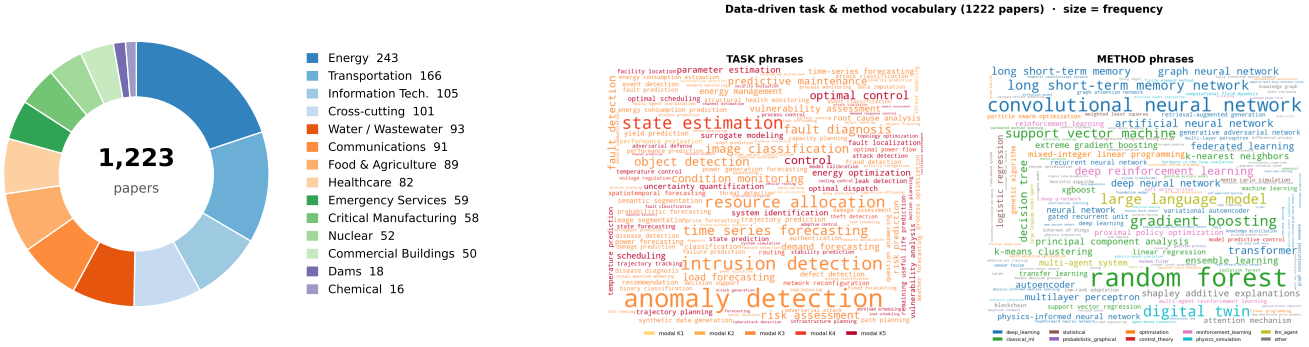

Figure 5: The literature corpus. Left: sector distribution. Right: data-driven task and method vocabulary; word size is corpus frequency.

Task class $\langle \omega, \tau, e \rangle$.
Output mode ω — **Inform**: report information about the target; **Act**: select an intervention intended to affect the target.
Target time τ — **Now** (t): the current system; **Future** $(t + \Delta)$: an outcome at a later time.
Target element e — observable **signals** y; latent **states** x; system **mechanisms** m.

Harness level K **(highest level materially used).**
K_1**: Model-only reasoning.** The method uses only the task input and model parametric knowledge; K_2**: Static knowledge.** + manuals, SOPs, specifications, design documents, or rule bases; K_3**: Temporal observations.** + telemetry, logs, alarms, or time-series measurements; K_4**: Structure and physics.** + topology, component relations, governing equations, physical constraints, or control logic; K_5**: Forward simulation.** + executable simulation, digital-twin rollouts, counterfactual evaluation, or optimization over hypothetical future trajectories.

Figure 6: Abridged annotation rubric aligned with the task and harness spaces defined in Section 4.

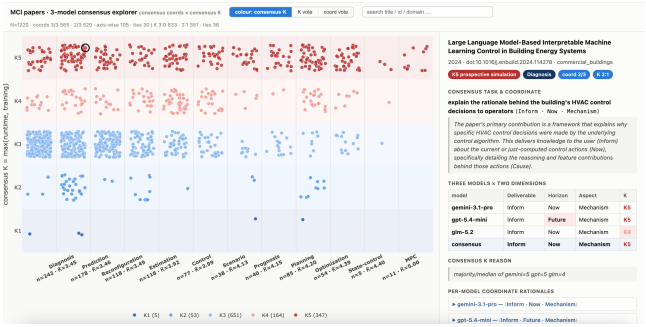

Figure 7: Snapshot of the interactive literature explorer to be released with the project. Each point represents one paper, grouped by task class and consensus harness level. The inspection panel presents the selected paper's metadata, supporting evidence, individual annotations, and consolidated labels.

Table 4: Consensus composition and inter-annotator agreement.

	3:0	2:1	adjudicated	pairwise agreement
Task coordinate	46%	54%	30 papers	κ 0.49–0.84 by axis
Harness level	68%	29%	36 papers	exact 73–84%; QWK 0.61–0.78

The remaining cases, comprising 30 task-coordinate assignments and 36 harness-level assignments, undergo a separate adjudication pass assisted by Claude Fable 5. The adjudicator reviews the paper evidence without access to the original model votes, and the resulting labels are manually verified. Table 4 reports the consensus composition and inter-annotator agreement.

We will release the complete extraction through an interactive literature explorer, including each paper's task coordinate, harness level, supporting evidence, individual annotations, and consolidated labels. The explorer also supports inspection and filtering across task classes and harness levels; Figure 7 shows a snapshot.

A.4 Corpus-Level Findings

Figure 2 shows the harness distribution for each task class, while Table 5 reports the counts used to estimate $P_{\text{lit}}(K \mid T)$. Each task axis is associated with higher harness provision: the mean level increases by 0.45 from Inform to Act and by 0.39 from Now to Future, while the marginal mean rises from 3.53 for observable signals to 3.65 for system mechanisms and 4.06 for latent states. Provision is concentrated at K_3 and K_5, which account for 53% and 28% of the corpus, respectively, compared with 13% at K_4.

Figure 8 provides a joint view of these trends over the complete 2×2×3 task space. Harness provision generally increases toward Act, Future, and latent-state tasks, with the highest values concentrated around A-F-x and adjacent Future–Act classes. The variation is nevertheless not determined by any single axis: task classes sharing one or two coordinates can still exhibit different literature-derived harness distributions.

The corpus records the harness level used by prior methods rather than the level sufficient for a fixed agent and environment. Estimates for sparsely represented classes, particularly A-N-x ($n = 5$) and A-F-x ($n = 11$), are also less reliable. We therefore treat the

Table 5: Literature-derived harness distributions across the 12 task classes, based on the consensus labels of three annotators. Counts are reported by harness level, and task classes are ordered by mean K.

Task class $\langle \omega, \tau, e \rangle$	n	K_1	K_2	K_3	K_4	K_5	$\bar{K}$	Mode
I-N-y	254	1	3	208	13	29	3.26	K_3
I-N-m	242	2	27	132	22	59	3.45	K_3
I-F-y	178	0	2	127	14	35	3.46	K_3
A-N-m	118	0	9	65	21	23	3.49	K_3
I-N-x	118	0	1	44	36	37	3.92	K_3
A-N-y	77	0	0	28	22	27	3.99	K_3
I-F-m	38	1	2	9	5	21	4.13	K_5
I-F-x	40	0	0	16	2	22	4.15	K_5
A-F-m	85	1	9	8	21	46	4.20	K_5
A-F-y	54	0	0	13	7	34	4.39	K_5
A-N-x	5	0	0	1	1	3	4.40	K_5
A-F-x	11	0	0	0	0	11	5.00	K_5

literature-derived map as a prior rather than verified execution evidence. More broadly, the analysis provides a survey-scale account of how MCI O&M research has provisioned system information and capabilities across task classes.

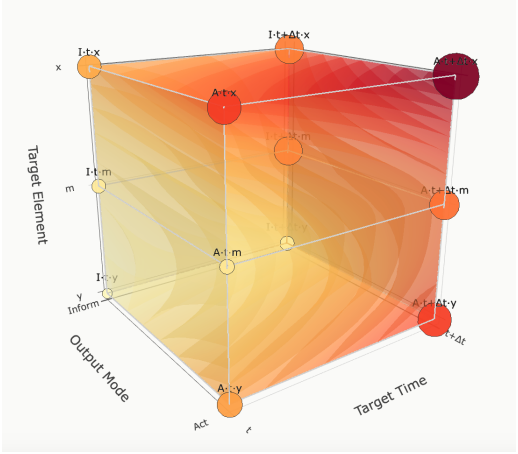

Figure 8: Three-dimensional view of the literature-derived harness landscape. The vertices represent the 12 task classes defined by output mode, target time, and target element, and surface colour indicates the mean literature-derived harness level $\bar{K}$. Conventional task names are shown only as approximate interpretations of the formal coordinates.

B Benchmark Construction and Verification

B.1 Simulation Environments

Liquid. A differentiable thermal-hydraulic digital twin of a 10-rack liquid-cooled data hall: one CDU loop (pump, control valve, facility heat exchanger) feeding 20 server cold plates over a 130-node/149-edge hydraulic network. Transient states integrate with `dopri5` at Δt=5 s (rtol 10^{-4}, atol 10^{-6}); steady states solve the coupled hydraulic–thermal system directly. An episode spans 0–1200 s. Telemetry is a ~910-column table of `family:target` signals (node temperatures and pressures, flows, pump speed, per-plate power); the internal ODE states (e.g., metal base-plate temperatures) are recorded but hidden from the agent, which grounds the latent-state (x) task classes.

Grid. A power-grid environment on Grid2Op's `l2rpn_case14_sandbox` (IEEE-14: 14 substations, 20 branches, 6 generators, 11 loads), with pandapower AC power flow as the K5 engine. Reference trajectories are 288-step days at 5-minute resolution with overflow disconnection disabled so that overloads remain observable. Telemetry is a 142-column table (per-line loading ρ, flows, currents, voltages, status; per-generator and per-load injections). Here latent-state tasks arise from *structurally absent* columns—network losses and the full bus-voltage vector are in no column, and three buses have no voltage sensor—rather than from masking.

B.2 Scenario Generation

Liquid uses six simulated trajectories (two normal-operation trajectories, single- and multi-plate overheating, a migrating hotspot, and a pump degradation ramp). Grid uses ten one-day windows selected by a two-pass scan over 1,004 Grid2Op chronics: two normal days, seven sustained-overload windows spanning three distinct bottleneck lines at increasing severity ($\rho_{\max}$ 1.00–1.26), and one mid-window line-trip fault that cascades to ρ=1.95 on a neighboring line. Every task binds a scenario file, an observation window, and a current time; the visibility gate (Appendix C) prevents future-peeking, so scenarios are replayable and deterministic.

B.3 Task Construction

Each domain instantiates 10 tasks for each of the 12 classes (240 total) from parameterized template families (e.g., `value_at`, `hidden_plate_temp`, `best_action_rho`, `min_pump_below_maxtemp`), with natural-language paraphrase applied on top of templates. Every task carries a machine-readable ground-truth specification `gt_spec`={method, args} resolved by a deterministic oracle (~80 methods for Liquid, 54 for Grid) with three evidence tiers: direct trajectory lookups, closed-form physics (e.g., $Q = \rho \dot{V} c_p \Delta T$), and simulator co-solves that call the *same* engine exposed to the agent at K5, with privileged access (full trajectory, no visibility gate, hidden states). Benchmark templates are written against the system representation, not against any literature paper, so the literature map and the benchmark share only the taxonomy.

B.4 Construction and Test Splits

Each class splits 5/5 into a construction (map-estimation) split and a held-out test split, stratified by template family so no family appears only in one split, with a fixed RNG seed. The full K_1–K_5 sweep runs on all 240 tasks, but execution-derived levels are estimated *only* from construction tasks and all policies are evaluated *only* on test tasks.

B.5 Programmatic and Human Verification

Programmatic checks: (1) the ground truth, re-fed verbatim to the grader, must score 1.0 on all 240 tasks; (2) a cross-split echo audit verifies that no test answer string appears in construction traces; (3) independent oracle re-derivation validates every stored ground

truth; (4) a probe pass removes degenerate axes discovered during construction (e.g., the slack generator's inert setpoint and a voltage-pinned bus in Grid, which would otherwise make some "counterfactual" tasks trivial or dead). Human review of sampled traces additionally corrected two Grid ground-truth values after an audit of the prediction/prognosis cells; the affected 20 tasks were re-executed at all levels before any map or policy result reported here.

B.6 Representative Tasks

Table 6 shows one test task per class from the Liquid domain (Grid analogues replace temperatures with line loadings and pump/valve actions with generation redispatch).

C Harness Implementations

C.1 Cumulative Harness Manifests

Levels are strictly cumulative ($\mathcal{H}_{K_1} \subset \cdots \subset \mathcal{H}_{K_5}$): each level's registry unions the previous level's tools, and an agent at level K is constructed with exactly that registry. Table 7 lists both manifests.

C.2 Tool and Data Interfaces

All tools are exposed as JSON function-calling schemas; results return as structured JSON (time series are capped at 100 points per call, transient rollouts at 18 steps). K5 override conventions are uniform (pump speed, valve opening, per-plate power in Liquid; generator/load MW and line outages in Grid). Simulator wall-time and call counts are metered separately per task.

C.3 Capability Enforcement

Capability boundaries are architectural, not prompt-level: a level-K agent's process is constructed with only that level's tool registry, so higher-level tools are not merely discouraged but nonexistent in its API schema. A shared visibility gate clamps every data access to the task's observation horizon (agent-supplied times are clamped server-side, including K5 rollout start times), and hidden state families are masked at the data source; the grading oracle deliberately bypasses both. During escalation, only the model's own textual findings are carried into the next attempt—no tool outputs, caches, or context cross the boundary.

C.4 Domain-Specific Differences

K1–K3 are semantically identical across domains (same tool names, same `kind:target` signal convention, same agent shell). K4 differs by physics content only (hydraulic network and heat balances vs. electrical topology, thermal limits, and power balances). K5 differs structurally: Liquid exposes a transient rollout because minute-scale thermal dynamics are genuine physics (plate thermal mass, exchanger lag), whereas the grid is quasi-static at 5-minute resolution—given injections, the state follows algebraically from power flow—so steady AC power flow plus N-1 screening *is* the complete simulation capability. A grid time-rollout is deliberately not exposed: the grid's future is driven by exogenous injection trajectories, so stepping the environment forward would either replay recorded future injections (leaking ground truth through the visibility gate) or reduce to the already-provided counterfactual solve under hypothesized injections; extrapolating those injections from telemetry is the agent's own K3-level task, not a harness capability. Latent state is gated telemetry in Liquid but structural sensor absence in Grid—two realistic mechanisms for the same taxonomy coordinate.

D Execution and Evaluation Protocol

D.1 Executor Configuration

All main results use a frozen `gpt-5.4` executor behind a ReAct-style function-calling loop: one model call per iteration, emitted tool calls executed and returned, until the model answers without tool calls or hits the iteration cap (20). Sampling: temperature 0.2, no output-token cap. Each task runs in an isolated spawned process with a 300 s hard timeout and a soft deadline 45 s earlier; on exhaustion the agent receives one final tool-free "answer from the evidence you have" call, so timeouts degrade to a best-effort answer rather than an empty one. The cross-executor study serves `Qwen3.5-27B` on a single-node vLLM server (thinking disabled, native tool-call parser, server-side prompt-tail truncation to fit the context window) behind the identical loop, harness, and tasks.

D.2 Task Scoring

Each task is scored in $[0, 1]$ as the mean of a deterministic rule score and an LLM-judge score. Rule graders are typed per task: numeric-with-tolerance (key-anchored extraction with unit normalization), set match (Jaccard), boolean polarity, directional, and required-fact containment. Inform tasks grade the reported estimate against the oracle value; Act tasks grade the selected intervention against the simulator-verified best action (e.g., the cheapest pump setting that satisfies the constraint), so action quality is grounded in the same physics the oracle uses.

D.3 LLM Judge

The judge is `gemini-3.1-pro` at temperature 0 with a fixed rubric (1.0 fully correct → 0.0 wrong or empty; credit substance over phrasing and unit differences). It sees only the question, ground truth, and final answer—never the harness level, policy, tool trace, or any experience text—so it cannot systematically favor a provisioning condition.

D.4 Cost Accounting

Token counts sum the API-reported usage of every executor-side LLM call in a condition—routing calls, self-checks, every escalation attempt, and verification calls for cascades—so policies are charged for their full mechanism; judge tokens are metered separately and never count toward policy cost. Latency is wall-clock per task, summed across attempts, and includes tool and simulator execution. Because absolute costs depend on the serving stack, the main text reports both normalized to Full-K_5 within each domain; absolute reference values appear in Appendix G.

D.5 Repetition and Randomness

Every condition runs three independent repetitions; tables report the mean ± population std of the three run means. Scenarios are deterministic replays, so nondeterminism enters only through model

Table 6: Representative Liquid benchmark tasks, one per class. S/X/M denote signal(y)/state(x)/mechanism targets(m); graders are deterministic rule checks combined with an LLM judge (Appendix D).

Class	Task	Question (abridged)	Grading
I-N-y	I-N-S-01	What is the outlet water temperature at cold plate `rack_a4_s1` right now?	numeric, ±0.6 K
I-N-x	I-N-X-01	How hot is the (unsensored) metal base plate of `rack_a4_s1` right now?	numeric, ±1.5 K
I-N-m	I-N-M-02	Outlets run high on both sides of the hall; which single device is operating far outside its typical range?	set match
I-F-y	I-F-S-01	The return-header temperature is drifting; using data up to now, where will it be at t=650 s?	numeric, ±0.4 K
I-F-x	I-F-X-02	`rack_a3_s2` steps from 503 W to 1400 W at $t\approx$305 s; what will its base-plate temperature reach?	numeric, ±1.5 K
I-F-m	I-F-M-03	If the pump were raised to 4500 rpm at today's load, where would the hottest outlet settle?	numeric, ±0.8 K
A-N-y	A-N-S-02	SLA: all outlets ≤300 K. Does the hall need an intervention right now?	boolean
A-N-x	A-N-X-02	Policy: every base plate ≤310 K (not directly measurable). Is an intervention needed now?	boolean
A-N-m	A-N-M-01	One chip is overheating; which cold plate do you act on?	set match
A-F-y	A-F-S-01	Lowest pump speed in {3000..6000} rpm keeping the hottest outlet within 305 K?	numeric, ±1 rpm
A-F-x	A-F-X-03	If the pump goes to 5500 rpm now, how much lower is `rack_a3_s2`'s base plate a minute later vs. no action?	numeric, ±0.6 K
A-F-m	A-F-M-01	Choose one measure to cool the hall: raise pump to 3500 rpm, migrate half of one rack's load, or open the valve.	set match

Table 7: Resources exposed at each harness level. Tools are OpenAI function-calling interfaces; each level unions all lower levels.

Level	Liquid	Grid
K1 model-only	no tools; parametric reasoning over the task prompt	no tools; parametric reasoning over the task prompt
K2 static	`static_knowledge` (catalog, parameter schemas, topology summary, typical ranges, units); `web_search`	`static_knowledge`; `search_knowledge` and `search_sop` (RAG over references and operator playbooks); `web_search`
K3 temporal	`list_signals`, `get_signal_window`, `compute_signal_stats`, `find_events` over gated telemetry	same four tools over the 142-column grid trajectory
K4 structure/physics	`query_topology`, `get_fluid_properties`, `check_physics` (energy balance, pressure drop, mixing), `check_constraints`	`query_topology`, `get_component_limits`, `check_physics` (power balance, loading, per-unit V), `check_constraints`
K5 simulation	`simulate_steady` (hydraulic/coupled), `what_if`, `compute_sensitivity`, `simulate_forward` (transient rollout)	`run_powerflow` (AC, with overrides), `what_if`, `contingency_analysis` (N-1), `compute_sensitivity`

sampling. Execution-derived levels are estimated from the pooled construction-split runs; policies are evaluated on the disjoint test split.

E Map Estimation and Statistical Analysis

E.1 Literature-Derived Map

We set $A_{\text{lit}}(T,K) = P_{\text{lit}}(K \mid T)$ and $\epsilon_{\text{lit}} = 0.05$. Under this tolerance, $\mathcal{G}_{\text{lit}}$ selects the modal harness level for all 12 task classes.

E.2 Execution-Derived Map

For each task class T, $\mu_{\text{exec}}(T,K)$ is the mean score over the five construction tasks and three runs. The execution-derived map selects the lowest harness level whose score is within $\epsilon_{\text{exec}} = 0.05$ of the best observed level. Table 9 reports the selected levels and their stability.

E.3 Tolerance Sensitivity

Table 8 sweeps ϵ_{exec}. Grid floors are invariant for $\epsilon \le 0.05$ and Liquid changes only two classes between 0 and 0.05; test accuracy is flat within noise across 0.05–0.15 while token cost varies <10%. The paper's operating point ϵ=0.05 sits on this plateau, so no conclusion depends on the tolerance choice.

E.4 Execution-derived Level Stability

Table 9 resamples the five construction tasks per class (1,000 bootstrap draws) and reports the probability of re-selecting the adopted floor, plus the set of floors reachable by leave-one-task-out. In 19 of

Table 8: Sensitivity of the execution-derived map to ϵ_{exec}. "Δfloors" counts classes whose selected level differs from ϵ=0.05; accuracy/tokens are test-split lookup results.

	Liquid			Grid		
ϵ	Δfl.	Acc ± std	Tok	Δfl.	Acc ± std	Tok
0.00	2	0.659 ± 0.021	14,738	0	0.762 ± 0.023	11,829
0.02	2	0.659 ± 0.021	14,738	0	0.762 ± 0.023	11,829
0.05	0	0.670 ± 0.020	14,009	0	0.762 ± 0.023	11,829
0.10	3	0.673 ± 0.022	14,246	1	0.749 ± 0.024	12,253
0.15	4	0.671 ± 0.024	12,851	2	0.742 ± 0.035	13,406

Table 9: Execution-derived level stability at ϵ=0.05: bootstrap probability of re-selecting the adopted floor over construction tasks, and leave-one-out floor sets.

	Liquid			Grid		
Class	floor	P(floor)	LOO	floor	P(floor)	LOO
I-N-y	K3	0.70	K3	K4	0.91	K4
I-N-x	K3	0.99	K3	K5	0.99	K5
I-N-m	K3	0.99	K3	K3	0.99	K3
I-F-y	K5	0.35	K3/K4/K5	K3	0.75	K3/K4
I-F-x	K5	0.68	K4/K5	K5	0.73	K3/K5
I-F-m	K5	1.00	K5	K5	1.00	K5
A-N-y	K5	0.92	K5	K5	0.92	K5
A-N-x	K5	0.69	K3/K5	K3	0.88	K3
A-N-m	K3	0.82	K3	K5	0.98	K5
A-F-y	K5	1.00	K5	K5	0.99	K5
A-F-x	K2	0.70	K2/K3	K5	0.68	K3/K5
A-F-m	K5	0.92	K5	K5	1.00	K5

24 class×domain cells the adopted floor is re-selected with probability ≥ 0.7; instability concentrates where adjacent levels are within tolerance of each other (e.g., Liquid I-F-S, where K3/K4/K5 all solve the class), i.e., exactly where the choice is least consequential. No resampling ever moves a floor below K2.

E.5 Statistical Tests

Paired bootstrap over the 60 test tasks (per-task scores averaged over three runs; 10^4 resamples): on Liquid, Exec-Lookup vs. Full-K_5 gives Δ=+0.018, 95% CI $[-0.022, +0.061]$—statistically indistinguishable accuracy at 14% lower token cost, which is the claimed trade. On Grid, Exec-Lookup is genuinely below Full-K_5 (Δ=−0.044, CI $[-0.087, -0.008]$) and genuinely above Lit-Lookup (Δ=+0.098, CI $[+0.020, +0.183]$), confirming both that maximal provision stays accuracy-optimal on Grid and that execution evidence is needed to calibrate the literature prior.

E.6 Oracle Definitions

The Class Oracle selects, per class, the level with the highest test-split mean; the Task Oracle selects the best level per individual task; both use mean scores across the three runs. Both peek at test outcomes and are therefore non-deployable ceilings; neither has a consistent cost, so they appear as accuracy lines only.

F Baseline Implementations

F.1 Full-K_5

The frozen executor with the complete K5 registry on every task; no routing calls.

F.2 Instruction–Tool Retrieval

The official ITR package with default configuration (hybrid dense + BM25 + cross-encoder retrieval). The tool corpus is the 14–16 harness tool specs verbatim; the instruction corpus is the K5 system prompt in fragments, with exemplars from successful construction traces (Grid). Per task, retrieved tools map to the minimal cumulative level containing them (retrieval confidence < 0.7 falls back to full provision); routing itself costs zero LLM tokens.

F.3 LLM-Route and LLM+Exp

One tool-free routing call to the same executor, presented with a generic legend of the five levels and instructed to pick the cheapest sufficient level, terminating in `LEVEL: Kx`. LLM+Exp additionally shows the router a distilled experience playbook: per-class recipe cards distilled by the executor from construction-split trajectories (multi-run scores, best tool sequences with observation snippets, contrastive failures; the pre-registered floor annotations are withheld), synthesized into a ≤1,200-word global playbook. The playbook informs routing only; execution runs on the raw task. Routing tokens count toward cost.

F.4 AutoMix

The official AutoMix implementation mapped from a model cascade to a harness cascade: the K3 episode plays the small model, the K5 episode the large one. Confidence uses the official few-shot self-verification prompt at temperature 1.0 with k=8 samples; routing uses the official 8-bin POMDP meta-verifier trained on construction-split rows (threshold fallback), with per-domain costs set to measured mean tokens. Verification tokens count toward cost, which is why AutoMix is expensive despite reusing episodes.

F.5 Blind-ESC

Identical loop, self-check protocol, one-tier-per-failure climbing, finding-carrying, and grading as Map-ESC; the only difference is the seed (K_1 vs. the mapped floor). It therefore isolates the value of the map from the value of escalation.

F.6 Reflexion and ExpeL

Reflexion runs at fixed K5 with the official reflection prompts and at most three attempts; since benchmark rewards are hidden at run time, the retry trigger is a ground-truth-free LLM self-verdict (PASS/FAIL), and reflections read the real tool trace. ExpeL runs at fixed K5 with the official pipeline: 20 induced rules from construction trajectories plus 6 task-similar few-shot exemplars retrieved by sentence-embedding kNN from 50 successful construction traces.

Table 10: Full K_1–K_5 sweep, all 120 tasks per domain, 3 runs.

	Liquid			Grid		
Level	Acc ± std	Tok	Lat (s)	Acc ± std	Tok	Lat (s)
K1	0.253 ± 0.002	346	3.5	0.191 ± 0.016	1,021	4.0
K2	0.216 ± 0.011	866	4.3	0.197 ± 0.004	2,012	4.5
K3	0.581 ± 0.008	11,617	13.7	0.502 ± 0.020	14,716	13.6
K4	0.560 ± 0.013	16,923	16.6	0.566 ± 0.011	18,179	12.8
K5	0.680 ± 0.015	17,794	50.2	0.773 ± 0.019	13,265	7.7

G Full Experimental Results

G.1 Aggregate K_1–K_5 Sweep

Table 10 gives the full-corpus sweep (all 240 tasks × 3 runs). Aggregate performance is not strictly monotonic, but the aggregate is exactly what the per-class analysis shows to be misleading: In Liquid, K_4 performs below K_3 in aggregate despite improving performance for several individual task classes. In Grid, K_5 is the most accurate level and uses fewer tokens than K_3 and K_4, although it is not the least costly level overall.

G.2 Per-Class Execution Scores

Table 11 reports the mean accuracy for all ten tasks in each class under K_1–K_5. These full-benchmark results complement, rather than reproduce, the construction-split scores shown in Figure 3.

G.3 Construction- and Test-Split Analysis

Figure 3 reports construction-split scores, from which the execution-derived map is estimated. All provisioning policies are evaluated on the disjoint test split in Table 1. Re-estimating the map on the test split changes the selected level in 10 of the 24 class–domain pairs, primarily in the classes identified as less stable in Table 9. Using the construction-split map results in a mean class-level test regret of 0.036, consistent with the gap between Exec-Lookup and the Class Oracle.

G.4 Absolute Reference Costs

Normalization anchors for Table 1 (test split, Full-K_5): Liquid 16,210 tokens and 47.2 s per task; Grid 13,409 tokens and 7.6 s per task. Multiplying Table 1 ratios by these anchors recovers absolute costs; e.g., Exec-Lookup averages ≈14.0k tokens/39 s on Liquid and 11,829 tokens/8.6 s on Grid.

H Additional Analyses

H.1 Routing Granularity and Construction Bias

The class is the right routing unit for this corpus: family-level routing (floors per template family) reaches 0.634/0.704 (Liquid/Grid) and task-level nearest-neighbor routing 0.639/0.690, both below class-level lookup (0.670/0.762). Preferred harnesses also vary *across* families within 7 of 12 Liquid and 5 of 12 Grid classes, so class-level floors are not an artifact of any single template family; they smooth over family-level sampling noise that finer-grained routing overfits.

H.2 Escalation Diagnostics

Map-ESC escalates rarely: 11.7% of Liquid and 13.9% of Grid test executions trigger the K_5 retry (mean 1.12/1.14 attempts per task). Seeds follow the map (Liquid: 105/180 executions start at K_5, 60 at K_3, and 15 at K_2.), and most below-K5 seeds finish where they started (Liquid: 39 of 60 K3-seeded executions; Grid: 20 of 45)—the self-check acts as tail-risk insurance on a calibrated start, not as a router.

Step-size ablation: is the direct K_5 jump too coarse? We ran the gradual alternative—$K_{i+1} = \min(K_i{+}1, K_5)$, findings carried at every hop—for three repetitions in both domains. It is dominated: accuracy falls to 0.692 ± 0.009 (Liquid) and 0.762 ± 0.010 (Grid) versus 0.715 ± 0.014 and 0.782 ± 0.010 for the direct jump, at equal cost on Liquid and higher cost on Grid (+5% tokens, +8% latency). The gradual paths explain why: of the K3-seeded executions that escalated, only 4 of 19 (Liquid) and 1 of 24 (Grid) were satisfied by the intermediate K4 step—the rest climbed on to K_5 anyway, having spent an extra attempt at still-insufficient provision and carried a failed intermediate trace into the final context. The insufficiency that survives a correctly-mapped seed is rarely one tier deep, so the intermediate grant buys little; escalating directly to full provision is simultaneously more accurate and more efficient, which is why Map-ESC uses the two-point design.

H.3 Alternative Experience Representations

Consistent with the main text, execution experience helps most as a *measurement* (the map) rather than as text: injecting the distilled playbook into execution at Full-K_5 helps Grid (0.810 vs. 0.806) but not Liquid (0.648 vs. 0.652), and routing-only use of the playbook (LLM+Exp) does not beat the free lookup in either domain. ExpeL's trajectory retrieval is the strongest execution-side use of experience on Liquid (0.731) but costs 44% more tokens than Map-ESC for a comparable gain.

H.4 Qualitative Error Analysis

Four recurring failure modes: (i) *under-provisioning*: below the floor, the agent either abstains or extrapolates from typical ranges (Liquid I-N-S at K1/K2 scores 0.00—the honest failure the taxonomy predicts); (ii) *over-provisioning backfire*: with simulation available, agents run counterfactuals instead of reading the trend (Liquid I-N-X drops from 0.59 at K3 to 0.46 at K5; Grid A-N-X from 0.69 to 0.65), and sensitivity sweeps burn the iteration budget; (iii) *executor-limited classes*: Liquid A-F-X never exceeds 0.35 at any level—transient counterfactual deltas exceed what the executor can orchestrate, and no provisioning policy can fix this; (iv) *self-check errors*: false passes dominate false escalations, which is why Blind-ESC stalls below K5 on classes whose confident-but-wrong answers never trigger the retry.

I Artifact Availability and Responsible Deployment

I.1 Released Artifacts

We will release both 120-task corpora with ground-truth specifications and splits, the frozen scenario telemetry (CSV), the harness manifests and tool schemas for all five levels, the literature corpus

Table 11: Per-class mean accuracy ± std for every harness level (10 tasks/class, 3 runs).

	Liquid					Grid				
Class	K1	K2	K3	K4	K5	K1	K2	K3	K4	K5
I-N-S	.00±.00	.00±.00	.90±.00	.88±.06	.87±.03	.03±.02	.03±.02	.78±.02	.90±.00	.88±.02
I-N-X	.02±.02	.04±.03	.59±.05	.53±.05	.46±.10	.00±.00	.00±.00	.23±.06	.63±.05	.90±.02
I-N-M	.36±.06	.29±.07	.90±.00	.87±.05	.85±.07	.23±.02	.26±.01	.93±.05	.93±.05	.93±.05
I-F-S	.15±.00	.15±.00	.70±.05	.65±.07	.71±.02	.10±.00	.12±.02	.42±.02	.45±.05	.44±.10
I-F-X	.20±.03	.14±.02	.30±.06	.33±.01	.36±.06	.15±.04	.10±.02	.26±.06	.32±.07	.36±.04
I-F-M	.32±.05	.31±.06	.34±.04	.53±.04	.78±.09	.10±.04	.12±.02	.30±.04	.29±.06	.88±.02
A-N-S	.28±.06	.24±.05	.55±.07	.45±.04	.73±.09	.30±.04	.28±.05	.61±.05	.61±.03	.95±.00
A-N-X	.53±.06	.42±.05	.67±.02	.60±.05	.72±.07	.40±.04	.39±.02	.69±.05	.55±.11	.65±.00
A-N-M	.12±.02	.25±.04	.83±.06	.82±.05	.85±.07	.37±.02	.35±.04	.75±.08	.80±.00	.97±.05
A-F-S	.22±.06	.17±.02	.27±.02	.15±.00	.75±.08	.26±.01	.22±.02	.27±.04	.33±.03	.83±.02
A-F-X	.23±.03	.28±.04	.27±.06	.19±.11	.23±.02	.20±.00	.17±.05	.37±.13	.40±.00	.49±.05
A-F-M	.62±.05	.30±.04	.65±.04	.73±.13	.83±.05	.15±.04	.33±.02	.40±.07	.57±.02	1.00±.00

IDs with all three per-model annotations and adjudications, the distilled experience playbooks, evaluation and map-estimation code, and per-task execution records (scores, token/latency/simulator meters, tool-call traces) sufficient to reproduce every table and figure without API access.

I.2 Proprietary Components

The liquid-cooling twin is currently proprietary. We will release its frozen trajectories, topology metadata, and recorded tool I/O, which are sufficient to reproduce all reported results. Generating new Liquid trajectories currently requires access to the twin; once it is open-sourced, we will release the complete Liquid environment. The Grid environment, based on Grid2Op and pandapower, is fully open-source.

I.3 Compute and API Usage

The two K_1–K_5 sweeps consumed 17.1M (Liquid) and 17.7M (Grid) executor tokens plus ∼2.4M judge tokens; policy and baseline conditions add a comparable amount, for ∼75M tokens total across all reported experiments. Qwen experiments ran on a single-node vLLM deployment; simulator compute is negligible (sub-second per solve, metered per task).